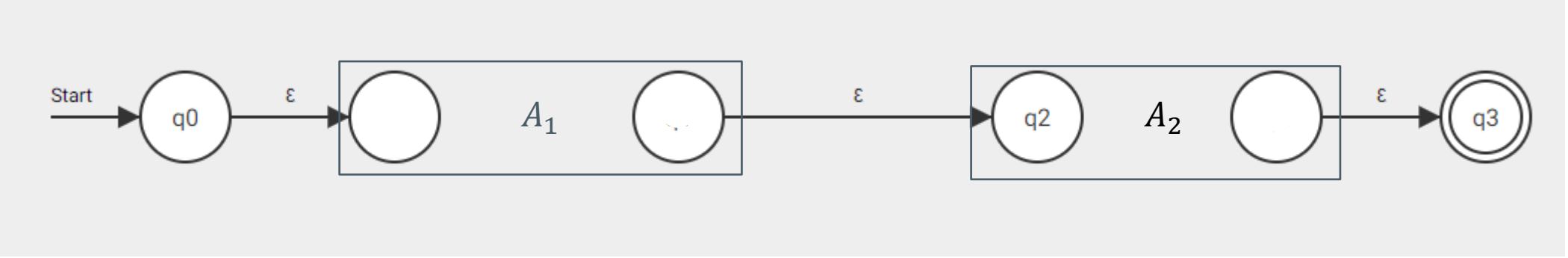
Abschlusseigenschaften regulärer Sprachen - Konkatenation

Satz: Die Klasse der regulären Sprache ist abgeschlossen unter Konkatenation.

Beweis:

Seien L_1 und L_2 reguläre Sprachen, die durch die NEA's A_1 und A_2 erkannt werden.

Wir verbinden die beiden Automaten sequentiell durch einen ϵ-Übergang:



Dieser neue Automat „rät“, wo der erste Teil des Eingabewortes $l_1 \in L_1$ endet, und erkennt dann $l_2 \in L_2$. In diesem Fall hat er $l_1 l_2 \in L_1 L_2$ erkannt.

Abschlusseigenschaften regulärer Sprachen – Vereinigung, Komplement, Schnitt

Satz: Die Klasse der regulären Sprachen ist abgeschlossen unter a) Vereinigung, b) Komplement und c) Schnitt.

Beweis:

a) Analog zum Beweis auf der Vorseite, kann man durch eine „Parallelschaltung“ zweier Automaten einen neuen Automaten konstruieren, der ein Wort w akzeptiert, wenn es in L_1 oder L_2 liegt.

b) Um zu zeigen, dass das Komplement einer Sprache L akzeptiert wird, ist zu zeigen, dass $\Sigma^* \setminus L$ akzeptiert wird.
Sei $M = (Q, \Sigma, \delta, q_0, F)$ der DEA, der L akzeptiert.
Dann akzeptiert $\overline{M} = (Q, \Sigma, \delta, q_0, Q \setminus F)$ das Komplement von L, da alle Zustände, in denen M akzeptiert, nun nicht mehr akzeptieren und umgekehrt.

c) Es gilt $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$.
Aufgrund von b) und c) ist der Schnitt zweier regulärer Sprachen regulär.

Reguläre Ausdrücke

Reguläre Sprachen können durch reguläre Ausdrücke beschrieben werden:

Beispiele:

```
"if"        // Buchstabe "i" gefolgt von Buchstabe "f"
"else"      //                    …
"("         // Das Zeichen für runde Klammer auf
")"         //                    …
"{"         //                    …
"}"         //                    …
""          // Kein Zeichen (= das leere Wort).
```

Das leere Zeichen, einzelne Zeichen und aneinandergereihte Zeichen sind reguläre Ausdrücke.

Zusammengesetzte Reguläre Ausdrücke

Definition:

a) Wenn $a \in \Sigma$, dann ist a ein regulärer Ausdruck.

b) ϵ ist ein regulärer Ausdruck.

Wenn r und s reguläre Ausdrücke sind, dann sind

c) (r | s) // r oder s

d) (rs) // r gefolgt von s

e) (r^+) // r beliebig oft hintereinander

f) (r*) // das leere Wort oder (r^+)

auch reguläre Ausdrücke.

Beispiele

Beispiele:

```
(ab)|(cd)        // Entweder ab oder cd, nicht aber abcd, ac, …
x*               // Leeres Wort oder x, xx, xxx, xxxx, ….
(a|b)*xy         // beliebig oft a oder b gefolgt von x und y
```

Aufgabe:

Erstellen Sie einen regulären Ausdruck für `cb, cbb, cbbb`

```
cb|cbb|cbbb     = c(b|bb|bbb)
```

Use Cases - 1

```
Uwe@DESKTOP-0O7TS1G MINGW64 ~/uwe/PI2TEST
$ ls
Aufgabe1.html  Aufgabe2.html  Aufgabe3.md  dummy  out.txt
```

Ermitteln Sie alle html-Dateien zu Aufgaben.

```
Uwe@DESKTOP-0O7TS1G MINGW64 ~/uwe/PI2TEST
$ ls Aufgabe?.html
Aufgabe1.html  Aufgabe2.html
```

File Globbing erlaubt es, Mengen von Dateinamen über wildcards to beschreiben.
? steht für ein Zeichen, * für einen beliebigen String.

Use Cases - 2

```
Uwe@DESKTOP-0O7TS1G MINGW64 ~/uwe/PI2TEST
$ ls
Aufgabe1.html  Aufgabe2.html  Aufgabe3.md  dummy  out.txt
```

Finden Sie alle Vorkommen des Strings „abs“ oder „aus“ in den Dateien.

```
Uwe@DESKTOP-0O7TS1G MINGW64 ~/uwe/PI2TEST
$ grep a[bu]s *
ausgabe.txt:-rw-r--r-- 1 Uwe 197121    0 Oct  8 18:25 ausgabe.txt
dummy:Haus
dummy:Habsucht
```

grep durchsucht die angegebenen Dateien (hier per File Globbing mit *) nach dem regulären Ausdruck (hier: a[bu]s).

Use Cases - 3

In einem Programm müssen Sie unterscheiden, ob der Benutzer einen Bezeichner (Identifier) eingegeben hat oder eine Zahl mit optionalem Vorzeichen.

```
String input = args[0].trim();

if (input.matches("[a-zA-Z_][a-zA-Z0-9_]*")) {
    System.out.println("Der eingegebene Wert ist ein Identifier.");
} else if (input.matches("[+-]?\\d+")) {
    System.out.println("Der eingegebene Wert ist eine Ganzzahl mit optionalem Vorzeichen.");
}
```

```
// boolean matches(String regex)
// Tells whether or not this string matches the given regular expression.
```

Use Cases – 3 : Alternative

Mächtiger als String.matches() ist das java.util.regex Package.

Beispiel:

```
// instantiate Pattern and Matcher
    Pattern p   = Pattern.compile(regex);
    Matcher m = p.matcher(text);

    // find all matches
    int anzahlMatches = 0;
    while (m.find()) {
        anzahlMatches++;
        int start = m.start();
        int end = m.end();
        System.out.println("Match found from " + start + " to " + end);
    }
    …
```

Aufgabe

In einem Java-Programm wollen wir prüfen, ob ein eingegebener Text dem Format einer eMail-Adresse entspricht.

Beispiel: heinz.otto.mustermann@Mail.exampleDomain.com

Geben Sie einen regulären Ausdruck an!

```
[\w\.-]+@[a-zA-Z\d\.-]+\.[a-zA-Z\d.]{1,}
```

Genauer:

```
[a-z0-9!#$%&'*+\/=?^_`{|}~-]+(?:\.[a-z0-9!#$%&'*+\/=?^_`{|}~-]+)*@(?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\.)+[a-z0-9](?:[a-z0-9-]*[a-z0-9])?
```

Aufgaben

a) Erstellen Sie einen regulären Ausdruck, der folgende Wörter erzeugt:

- a gefolgt von einem oder vielen b‘s oder
- c gefolgt von keinem oder vielen a‘s

Lösung: $ab^+ \mid ca^*$

b) erstellen Sie einen regulären Ausdruck, der die Menge aller Binärzahlen erzeugt.

Lösung: $(0|1)^+$

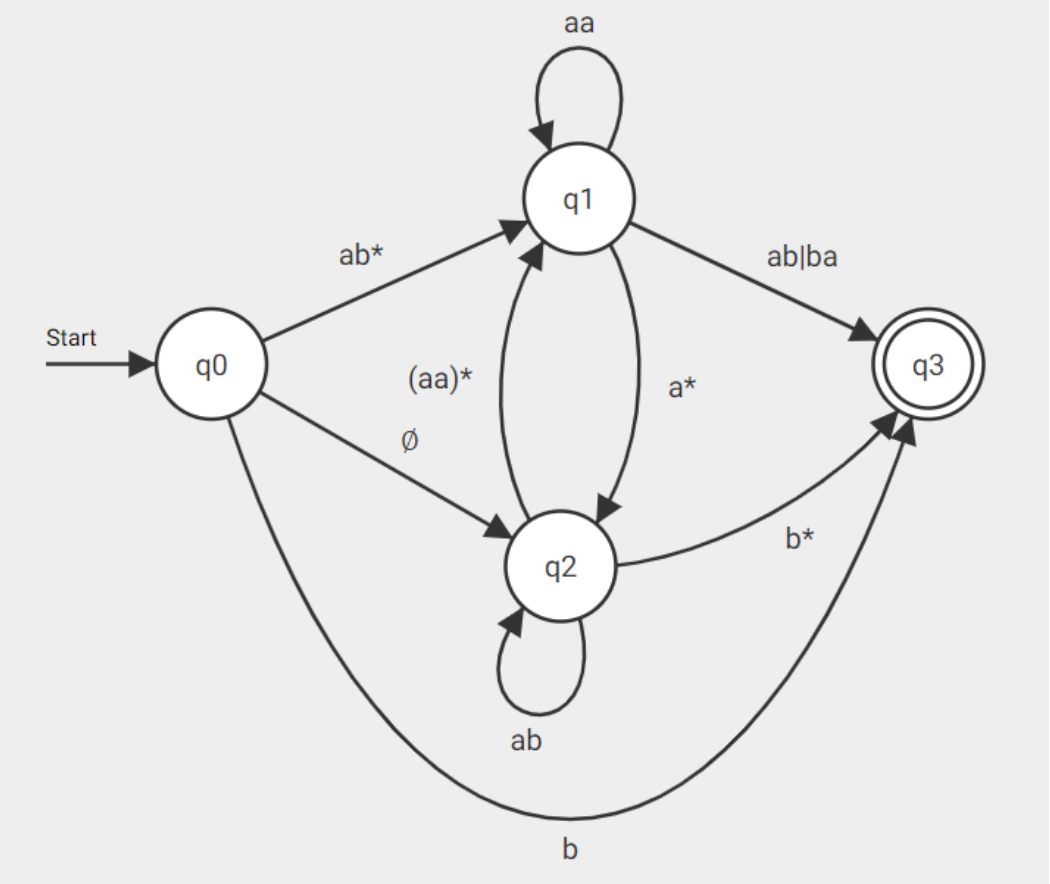
c) erstellen Sie einen regulären Ausdruck, der die Menge aller Binärzahlen mit einer geraden Anzahl von Stellen erzeugt.

Lösung: $(00|01|10|11)^+$

Generalisierte nichtdeterministische endliche Automaten (GNEA)

- GNEAs sind NEAs, bei denen die Übergänge nicht bei einzelnen Zeichen (oder ϵ) , sondern regulären Ausdrücken erfolgen.

- Der Einfachheit halber verlangen wir:
  - Der Startzustand hat eine Verbindung zu jedem anderen Zustand, aber keine eingehenden Kanten
  - Es gibt nur einen Endzustand und dieser hat keine ausgehenden Kanten.
  - Startzustand und Endzustand sind verschieden.
  - Alle anderen Zustände haben Kanten zu jedem anderen Zustand und sich selbst.



DEA -> GNEA

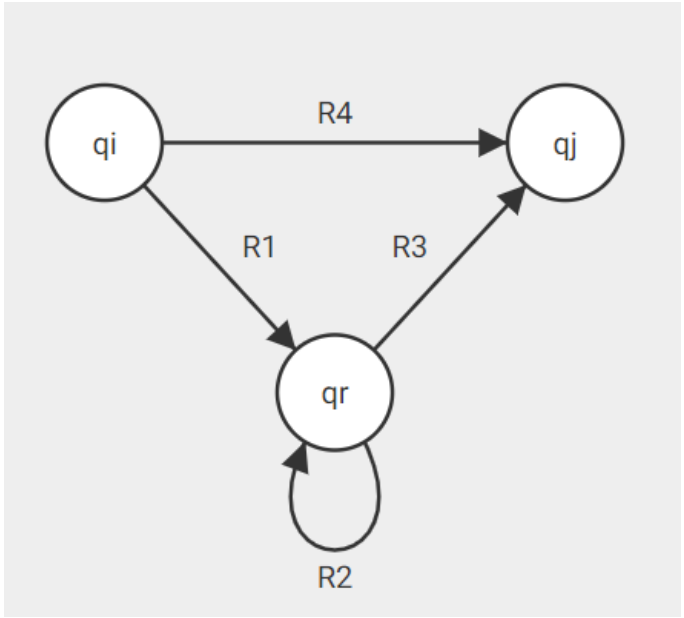
Man kann DEA‘ in GNEAs umwandeln:

- Neuen Startzustand hinzufügen und per ϵ –Übergang mit dem vorigen Startzustand verbinden.
- Neuen Endzustand hinzufügen mit Übergängen von den vorigen Endzuständen.
- Wenn es Kanten mit mehreren Markierungen $a_1, \dots, a_n$gibt, ersetze diese durch eine Kante mit $a_1| \dots |a_n$.
- Sind zwei Zustände nicht miteinander verbunden, füge eine Kante beschriftet mit $\emptyset$ hinzu.

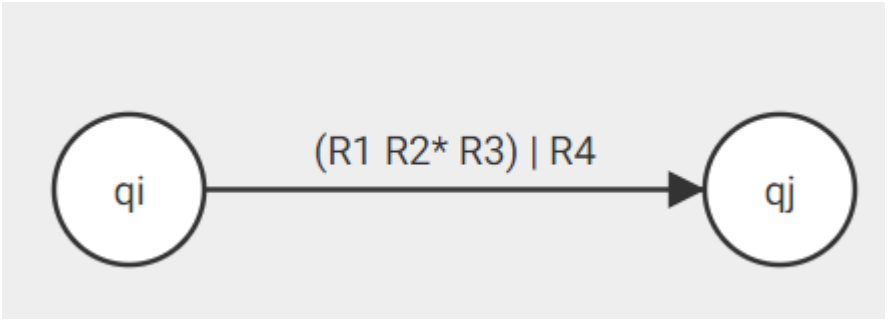
Alle diese Transformationen ändern die durch den Automaten erkannte Sprache nicht.

GNEA -> Regulärer Ausdruck

- Um zu zeigen, dass man jeden GNEA in einen regulären Ausdruck überführen kann, nehmen wir an, dass der GNEA k Zustände hat. Es muss gelten $k \geq 2$. (Warum?)
- Wenn $k > 2$, löschen wir immer wieder einen Zustand, solange bis nur noch Start- und Endzustand übrig bleiben:







- Der reguläre Ausdruck an der verbleibenden Kante vom Start- zum Endzustand ist der gesuchte reguläre Ausdruck.

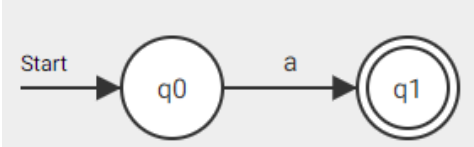
Äquivalenz mit endlichen Automaten – Teil 1

Satz: Eine Sprache ist regulär, wenn sie durch einen regulären Ausdruck beschrieben wird.

Beweis:

Wir beweisen zuerst, dass eine Sprache, die durch einen regulären Ausdruck beschrieben wird, regulär ist, indem wir für die Basisfälle einen Automaten angeben:

a) $a \in \Sigma$



b) Für $r = \epsilon$ wird der NEA



c) – f) folgen aus der Abgeschlossenheit regulärer Sprachen unter Vereinigung, Schnitt und Konkatenation.

Äquivalenz mit endlichen Automaten – Teil 2

Satz: Wenn eine Sprache regulär ist, dann wird sie durch einen regulären Ausdruck beschrieben wird.

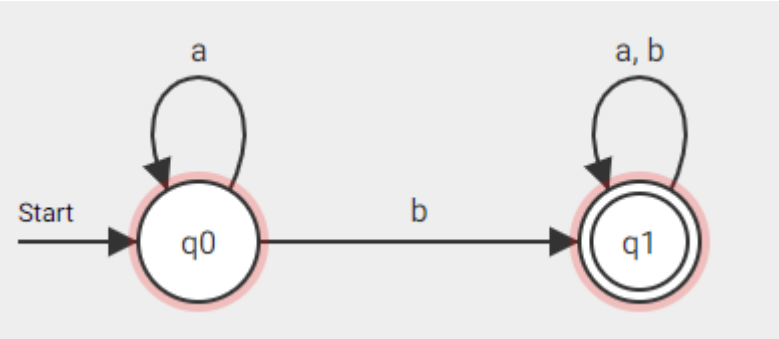
Beweis:

(i) Wenn eine Sprache A regulär ist, dann wird sie durch einen DEA erkannt (Definition von regulär).

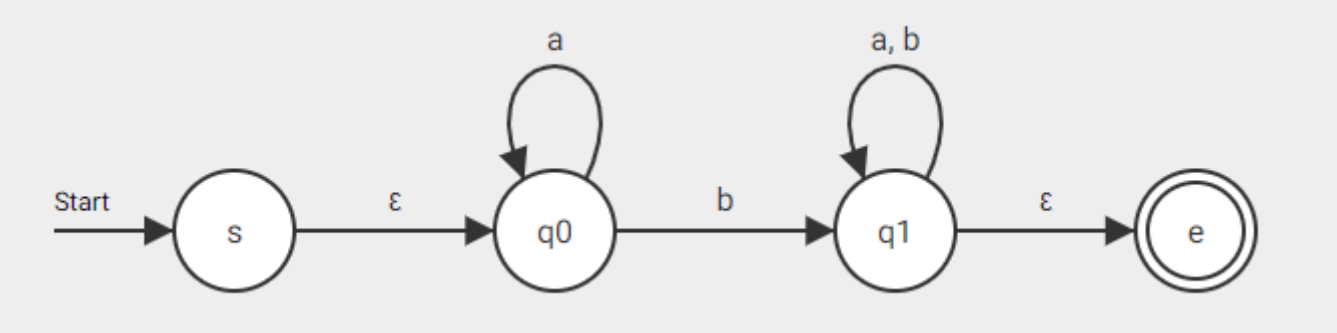
(ii) Zu jedem DEA gibt es einen äquivalenten GNEA (siehe Konstruktion eines GNEA)

(iii) Jeder GNEA kann in einen äquivalenten regulären Ausdruck umgeformt werden (siehe „GNEA->regulärer Ausdruck“)

Beispiel (1/2)

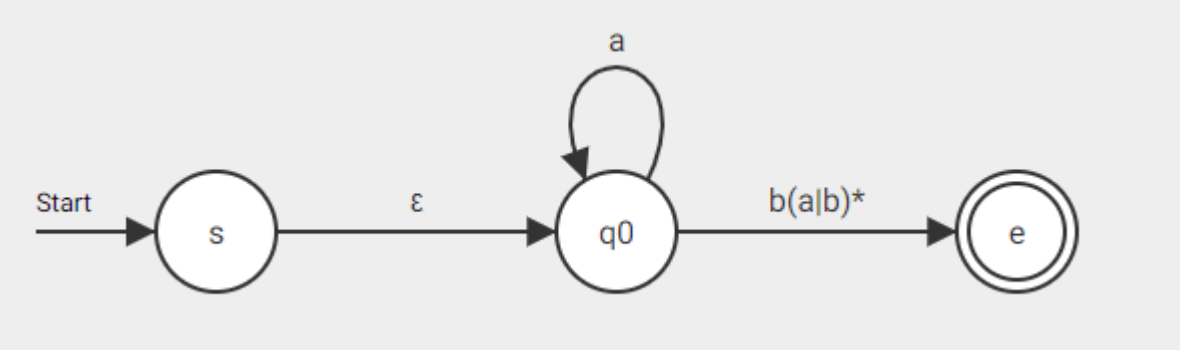


Schritt 1: Hinzufügen Start- und Endzustand

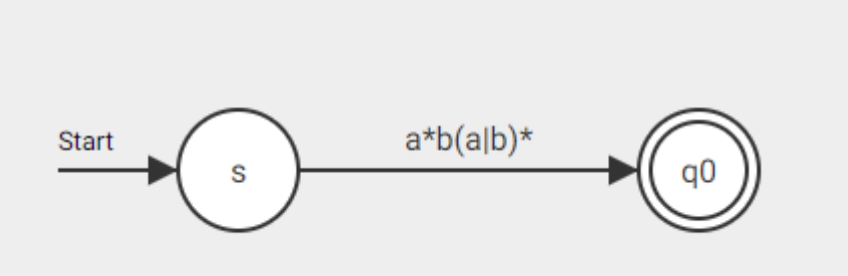


Beispiel (2/2)

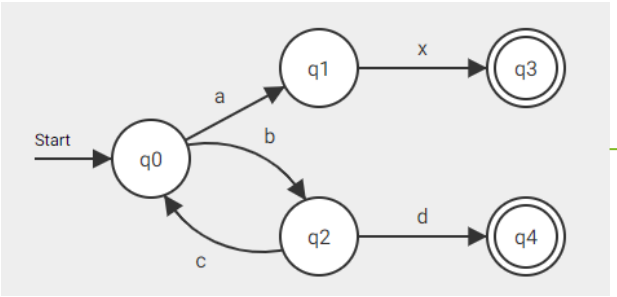
Schritt 3: Entfernen q1



Schritt 4: Entfernen q0

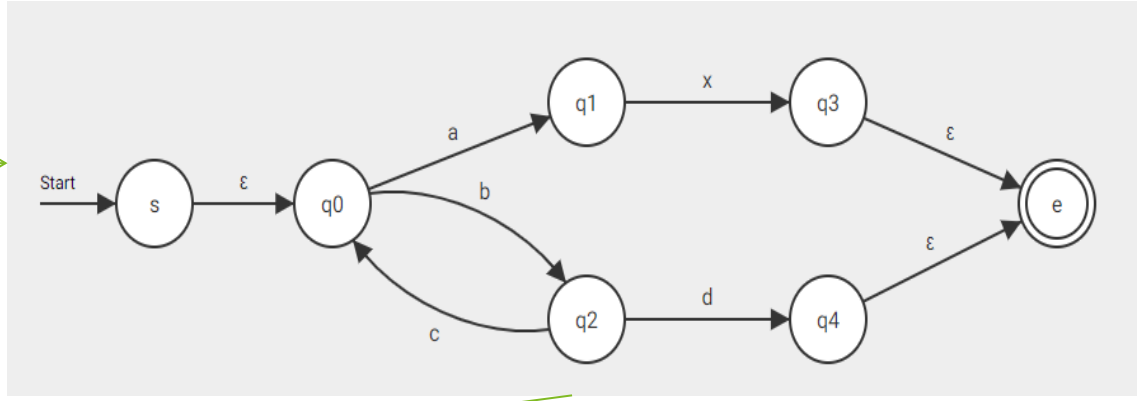


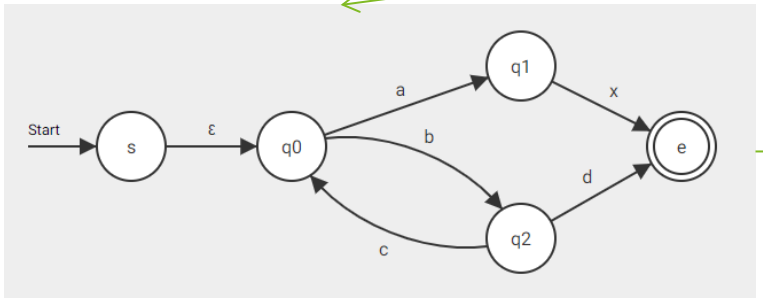
Tafel-Beispiel



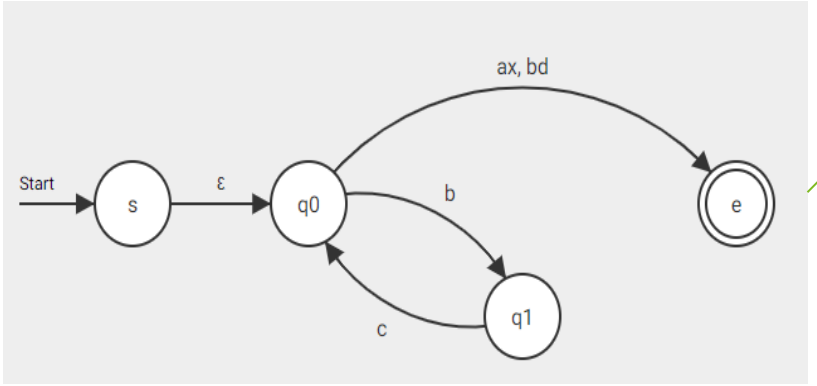
Ausgangs-DEA

Äquivalenter GNEA (ohne ø-Kanten)



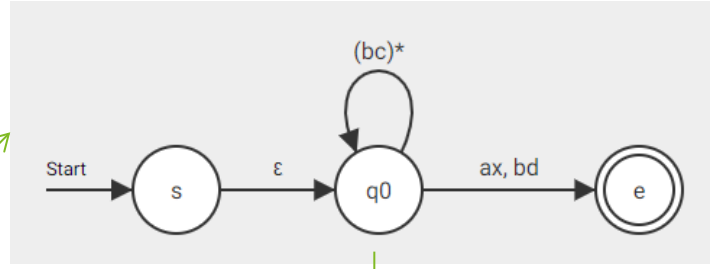


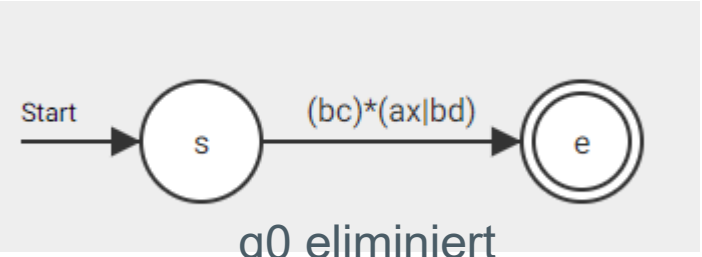
q3, q4 eliminiert



q1 „halb“ eliminiert

q1 eliminiert





q0 eliminiert

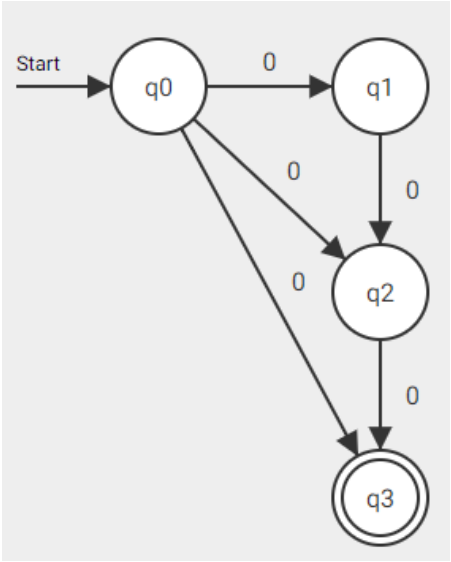
Pumping Lemma für reguläre Sprachen

Wir betrachten einen Automaten mit n Zuständen $A = (Q, \Sigma, \delta, q_0, F)$ *mit* $|Q| = n$ *und* $|\Sigma| = 1$.

Sei $L(A) = \{w \in \Sigma^* : 1 \leq |w| < n\}$, d.h. $L(A)$ enthält nur Wörter kürzer als n.

Beispiel: $Q = \{q0, q1, q2, q3\}$, $\Sigma = \{0\}$, $F = \{q_3\}$

$L(A) = \{0, 00, 000\}$



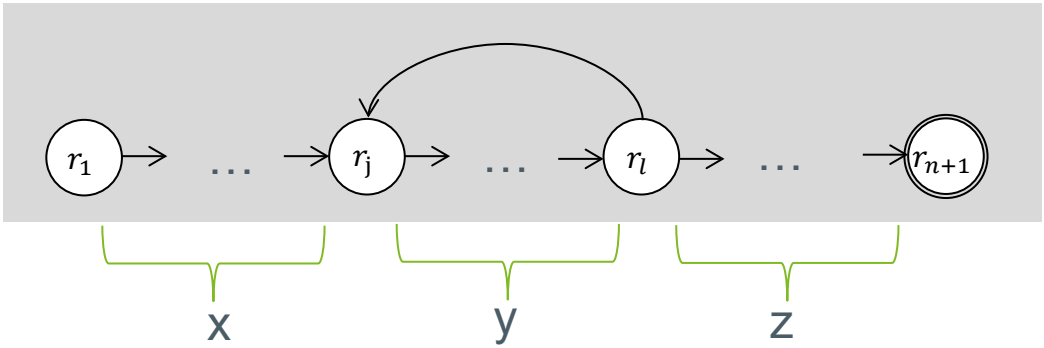
Pumping Lemma für reguläre Sprachen (II)

Jeder endliche Automat hat nur endlich viele Zustände (sic!).

Was passiert also bei regulären Sprachen, in denen es Wörter länger oder gleich $|Q|$ gibt?

Beim Erkennen eines Wortes mit Länge $\geq n$ muss mindestens ein Zustand mehrmals durchlaufen werden.

Pumping Lemma für reguläre Sprachen (III)



y sei das Teilwort, das in der „Schleife“ erkannt wird, u das Teilwort davor und w das Teilwort danach.

Beobachtung: Wenn das Wort xyz zur Sprache gehört, dass muss das Wort $xyyz$ auch zur Sprache gehören (und xyyyz, …).

Pumping Lemma für reguläre Sprachen (IV)

Satz: Wenn L eine reguläre Sprache ist, dann gibt es eine Konstante p („Pumping Länge“), so dass für alle $s \in L$ mit $|s| \geq p$ eine Aufteilung $s = xyz$ mit

(i) $|y| > 0$

(ii) $|xy| \leq p$

(iii) $\forall i \geq 0 : xy^i z \in L$.

existiert.

Pumping Lemma für reguläre Sprachen (IV)

Beweis:

Sei $A = (Q, \Sigma, \delta, q_1, F)$ ein DEA, der die Sprache L erkennt. p sei die Anzahl der Zustände von A.

Sei $s = s_1 \dots s_n$ ein Wort in L der Länge n mit $n \geq p$ und seien $r_1, \dots, r_{n+1}$ die Zustände, die A durchläuft, wenn s verarbeitet wird (also $r_{i+1} = \delta(r_i, s_i)\ für\ 1 \leq i \leq n$).

Da $n \geq p$ muss $n + 1 \geq p + 1$ sein. D.h., dass mindestens zwei dieser Zustände gleich sein müssen; wir nennen diese r_j und r_l. Dann muss $l \leq p + 1$ gelten.

Definiere $x = s_1 \dots s_{j-1}$, $y = s_j \dots s_{l-1}$ und $z = s_l \dots s_n$.

A muss dann auch xy^iz für alle $i \geq 0$ erkennen.

Da $j \neq l$, muss $|y| > 0$ gelten.

Da $l \leq p + 1$ gilt, muss $|xy| = l - 1 \leq p$.

Damit sind die drei Bedingungen des Pumping Lemmas erfüllt.

Anwendung des Pumping Lemmas

Nochmal: Die Umkehrung des Satzes gilt nicht!

Der Satz ist ein reiner Existenzsatz, er macht keinerlei Angabe darüber, wie man p oder x, y, z findet.

Aber: Man kann ihn sehr gut benutzen, um mittels Widerspruchsbeweis zu zeigen, dass eine gegebene Sprache nicht regulär ist.